SPURVANCE ASSIGNMENT

File Integrity Monitoring (FIM)

Sudip Kumar Chaurasiya

College: Model Institute of Technology (MIT)

Organization: Spurvance

Date: October 12, 2026

Abstract

This assignment describes a small File Integrity Monitoring (FIM) project named SentinelHash. The project creates a SHA-256 baseline for three sample files, protects a separate copy of the baseline with GPG encryption, and compares the current files with the original baseline. The checker reports files that are added, modified, or deleted. Three controlled tests were completed in Kali Linux. The results showed that the checker detected each type of change. The sample files were restored afterward, and a final check reported that no changes were detected.

Introduction

File Integrity Monitoring is a method for checking whether files have changed. A cryptographic hash creates a fixed-length value from a file's contents. If the contents change, the hash will normally change too. By comparing a current hash with a trusted baseline hash, a monitoring script can identify possible additions, modifications, and deletions.

The purpose of this assignment was to build and test a small FIM tool in a controlled Kali Linux lab. The project uses Python for file hashing and comparison, a plain-text baseline for the original hashes, and GPG to create an encrypted copy of the baseline.

Method

Lab Environment and Sample Files

The project was completed in Kali Linux using Python 3 and GPG. The monitored directory was ~/FIM-Lab. The baseline was stored separately as ~/FIM-Lab-baseline.txt. The checker script was named fim_monitor.py. Three sample files were included in the original baseline.

Sample file	Original content	SHA-256 hash
critical.txt	approved configuration	30488ba9d1cf1884837d03a9b5e9e7108b49c8f28d0ddcc31df764a1149b631e
readme.txt	training sample	4ea2a72ed972663f8e56c9680a9e7e5dde08b2a4810e50f5737b88f5a2fc5cae
system.conf	firewall=enabled	f9906af395d0ee704a68d793b52136ba5a2539b58b92fb5187c9cf5e0a0a3127

Baseline Generation and Encryption

The baseline report recorded the path and SHA-256 hash of each sample file. This baseline acted as the trusted reference for testing. A separate encrypted copy was created with GPG using AES-256 encryption. The encrypted baseline was decrypted for verification and compared with the plain-text baseline. The comparison succeeded, confirming that the encrypted copy matched the original baseline.

Checker Design

The Python checker reads the baseline, scans files directly inside the FIM-Lab directory, calculates SHA-256 hashes, and compares the current state with the baseline. It reports ADDED when a file is not listed in the baseline, MODIFIED when a known file has a different hash, and DELETED when a baseline file is missing. The checker writes a timestamp, event, path, old hash, and new hash to a CSV report.

Results

Test 1: Added File

Action: A new file named new.txt was created in the monitored directory. The checker was run after the file was added.

Output: The checker reported ADDED for /home/kali/FIM-Lab/new.txt.

Meaning: The checker detected a file that was not present in the original baseline. This shows that the script can identify newly created files in the monitored directory.

Usefulness: Unexpected new files can indicate an unapproved change. An administrator can investigate the file and decide whether it is expected.

Test 2: Modified File

Action: Text was appended to critical.txt to simulate an unauthorized change. The checker was run again.

Output: The checker reported MODIFIED for /home/kali/FIM-Lab/critical.txt. It also reported new.txt because that file was still present.

Meaning: The changed contents produced a different SHA-256 hash from the baseline. The mismatch allowed the checker to identify the modified file.

Usefulness: Hash comparison can help detect unexpected edits to configuration files, scripts, or other files that should remain unchanged.

Test 3: Deleted File

Action: The baseline file readme.txt was deleted. The checker was run a third time.

Output: The checker reported DELETED for /home/kali/FIM-Lab/readme.txt. It also reported the modified critical.txt and added new.txt, which were still different from the baseline.

Meaning: The checker compared the baseline paths with the current directory and found that readme.txt was missing.

Usefulness: Missing files can indicate accidental deletion or unauthorized activity. The result provides a starting point for further investigation.

Restoration and Final Check

After the tests, critical.txt and readme.txt were restored to their original contents, and new.txt was removed. The checker was run again. The final output stated, No file changes detected. This confirmed that the monitored sample files matched the original baseline at the time of the final check.

The test-results CSV preserves the combined findings from the test run: ADDED for new.txt, MODIFIED for critical.txt, and DELETED for readme.txt. Screenshots document baseline creation, encryption verification, each test action and detection, restoration, and the final clean check.

Discussion

The project demonstrated a basic method for monitoring file integrity with SHA-256 hashes. The separate encrypted baseline provided additional protection for the reference data. The tests showed that the checker could identify the three required event types. The final clean check showed that the sample files had been restored to their expected state.

A hash mismatch indicates that file contents changed, but it does not prove that the change was malicious. A new or deleted file also requires context before its cause can be determined.

Limitations

The current version scans only files directly inside the monitored directory and does not scan subdirectories. The CSV report is overwritten on each run, so evidence from individual runs must be saved separately when a longer history is required. The baseline must be protected from unauthorized changes. If an attacker can change both the files and the baseline, the comparison may no longer be trustworthy.

The checker reports file changes but does not identify who made a change, why it occurred, or whether it is harmful. It does not provide automatic alerts or continuous background monitoring. These features could be added in a future version.

Conclusion

This assignment completed a basic File Integrity Monitoring project using Python, SHA-256, and GPG in Kali Linux. A baseline was created for three sample files, and an encrypted copy was verified. The checker successfully reported an added file, a modified file, and a deleted baseline file. After restoration, the final check reported no changes. This practical work improved understanding of file hashing, baseline protection, change detection, and evidence collection.

Tools Used

Kali Linux, Python 3, Python hashlib, GPG encryption, CSV reporting, and terminal screenshots.

AI Use Disclosure

AI assistance was used to help organize the report and improve wording. The lab setup, commands, file changes, checker runs, and screenshots were completed and verified by the student. The results are based on the recorded lab output.

Evidence Index

The submission includes the Python source code, plain-text baseline, encrypted baseline, test-results CSV, README, test explanations, and timestamped evidence screenshots. The screenshots document baseline creation, encryption and verification, the three tests, restoration of sample files, and the final integrity check.